INSTALL

BobaLink — Installation Guide

Revision: 1 **Last modified:** 2026-06-10T21:00:00Z **Scope:** Building, loading, and running BobaLink (`extension/`) for end users and operators. **Authority:** `extension/package.json`, `extension/wxt.config.ts`, `extension/ci-ext.sh`, `docs/browser_extension/Status.md` (Rev 4).

> Accuracy note (§11.4.6): every command below is exactly as declared in `extension/package.json` / `extension/ci-ext.sh`. Steps not yet shipped are marked **(planned)**.

Requirements

- **Node.js ≥ 20** (`extension/package.json` → `"engines": { "node": ">=20.0.0" }`).
- **npm** (ships with Node).
- For the artifact gate: **`jq`** on `PATH` (the `ci-ext.sh` pre-flight requires `node`, `npx`, and `jq`).
- A **running Boba merge service on `http://localhost:7187`** to actually send torrents. BobaLink only talks to `localhost:7187` (`host_permissions` in `wxt.config.ts`). The extension *builds* and *loads* without it, but detect→send needs the backend up. Start it from the repo root with `./start.sh` (see the project `CLAUDE.md` for the full Boba stack).
- Target browsers: Chrome ≥ 109 (`minimum_chrome_version: "109"`) and Firefox (the build produces a `firefox-mv2` artifact).

There is **no global install** of BobaLink — you build it from source and load it unpacked (the store-listing/submission path is **(planned)** — Phase 9).

Build

From the `extension/` directory:

```
cd extension
npm install            # installs deps; postinstall runs `wxt prepare`
npx wxt build          # → .output/chrome-mv3/   (Chrome MV3, the default)
npx wxt build -b firefox   # → .output/firefox-mv2/
```

Convenience npm scripts (declared in `extension/package.json`) wrap the same commands:

```
npm run build          # wxt build           (Chrome)
npm run build:firefox  # wxt build -b firefox (Firefox)
npm run zip            # wxt zip    → .output/bobalink-1.0.0-chrome.zip
npm run zip:firefox    # wxt zip -b firefox → firefox + sources zips
```

`npm run zip` / `npm run zip:firefox` produce the per-store packages (`bobalink-1.0.0-{chrome,firefox,sources}.zip`) used for manual submission.

Load unpacked in Chrome (and Chromium / Edge / Opera / Yandex)

1. Run `npx wxt build` so `extension/.output/chrome-mv3/` exists.
2. Open `chrome://extensions`.
3. Toggle **Developer mode** on (top-right).
4. Click **Load unpacked**.
5. Select the **`extension/.output/chrome-mv3`** directory.

BobaLink appears in the toolbar. The build emits a valid MV3 manifest (`manifest_version: 3`, `background.service_worker: background.js`, `popup.html`, `options.html`, `content-scripts/content.js`, icons 16/32/48/128, and `_locales/{en,ru}/messages.json`).

> The `_locales/<default_locale>/messages.json` catalog is **required** because the manifest sets `default_locale: "en"` and uses `__MSG_*__` placeholders — Chrome rejects the extension at load if it is missing. The `ci-ext.sh` gate asserts it is present (§11.4.38; see `Status.md` Rev 4 for the bluff-audit that caught a build which had dropped it).

Load in Firefox

1. Run `npx wxt build -b firefox` so `extension/.output/firefox-mv2/` exists.
2. Open `about:debugging#/runtime/this-firefox`.
3. Click **Load Temporary Add-on…**.
4. Select the **`manifest.json`** inside `extension/.output/firefox-mv2/`.

(Temporary add-ons are removed when Firefox restarts; reload after each rebuild.)

The Firefox build is MV2 (`firefox-mv2`) via WXT. Tab-group features are Chrome-only and degrade gracefully on Firefox.

The `ci-ext.sh` gate (manual release-readiness check)

`extension/ci-ext.sh` is the single, **manual** pre-release gate. Run it before packaging a release:

```
cd extension
bash ci-ext.sh
```

It runs, FAILing loudly at the first failing step:

1. **type gate** — `npx tsc --noEmit`
2. **lint gate** — `npm run lint`
3. **unit suite** — `npx vitest run` (exit-0 required; the count is **not** hardcoded)
4. **chrome build** — `npx wxt build`
5. **firefox build** — `npx wxt build -b firefox`
6. **§11.4.38 artifact verification** — opens the produced `.output/chrome-mv3/manifest.json` (not the source), asserts `manifest_version == 3`, the service worker is declared, and **every manifest-referenced asset plus the `_locales/<default_locale>/` catalog exists on disk and is non-zero** — including following each HTML page's local `src`/`href` references.
7. **per-store zips** — `npm run zip` + `npm run zip:firefox`, asserting each store zip is at least 10 KiB.

On success the final line is `CI-EXT: PASS`. The script touches **no git**, makes **no commit**, and starts **no container**.

NO CI/CD — manual only (permanent)

BobaLink has **no CI/CD pipeline of any kind** and none may be added:

- No `.github/workflows/`, no `.gitlab-ci.yml`, no Jenkins, no pipelines, no git hooks for this subproject (project `CLAUDE.md` “CI IS MANUAL”; plan §0.1; Boba Hard Stop §1).
- The reference materials shipped `ci.yml`/`release.yml` — those were **dropped**; their build/test/lint commands live only in the manual `extension/ci-ext.sh`.

All validation runs by hand via `bash ci-ext.sh` (and, at the repo level, the manual `./ci.sh`).

Verifying a working install

1. Build + load unpacked (above).
2. Start the Boba stack so the merge service answers on `http://localhost:7187`.
3. Visit a torrent page (e.g. one of the supported sites in `USER_GUIDE.md`).
4. Open the popup — detected magnets/`.torrent` links should be listed.
5. Click **Send** / **Send All**, or use a context-menu / keyboard action.

> The end-to-end “torrent then appears in qBittorrent” confirmation against a live `:7187` backend is **(in progress)** (Phase 4 — see `Status.md`). The build/load and on-page detection steps are shipped.